



INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DA PARAÍBA
CAMPUS CAMPINA GRANDE - PB

CONTROLE E AUTOMAÇÃO:
ESTUDO DIRIGIDO PARTE II

RAUL CONFESSOR OLIVEIRA SILVA

CAMPINA GRANDE - PB
2026

Etapa 2 - Modelagem e Simulação

1. Introdução

Nesta etapa o objetivo é pegar os conceitos teóricos da Etapa 1 e aplicar na prática, modelando sistemas físicos reais. A ideia é partir da descrição física do sistema, deduzir as equações e transformar que governam e transformam tudo em função de transferência para prosseguir a análise do comportamento.

Foram escolhidos dois sistemas clássicos da engenharia: o circuito RLC série e o motor de corrente contínua (motor DC). Os dois aparecem bastante em aplicações industriais e servem muito bem para entender como a teoria se conecta com a prática

2. Circuito RLC Série

2.1 Descrição Física

O circuito RLC série é formado por um resistor (R), um indutor (L) e um capacitor (C) conectados em série com uma fonte de tensão $u(t)$. A saída analisada é a tensão no capacitor, $y(t) = v_C(t)$

É um dos sistemas mais estudados em controle porque representa diretamente um sistema de segunda ordem, com comportamento que vai desde superamortecido até oscilatório, dependendo dos valores dos componentes

2.2 Equações Diferenciais

Pela lei Kirchhoff das tensões, a soma das quedas de tensão no circuito é igual à tensão de entrada:

$$u(t) = v_R(t) + v_L(t) + v_C(t)$$

Substituindo as relações de cada componente:

$$u(t) = R.i(t) + L.(di/dt) + v_C(t)$$

Como a corrente e a tensão no capacitor se relacionam por $i(t) = C.(dv_C/dt)$, substituindo:

$$u(t) = RC.(dv_C/dt) + LC.(d^2v_C/dt^2) + v_C(t)$$

Reorganizando:

$$LC.y(t) + RC.y(t) + y(t) = u(t)$$

2.3 Função de Transferência

Aplicando a Transformada de Laplace com condições iniciais nulas:

$$LCs^2.Y(s) + RCs.Y(s) + Y(s) = U(s)$$

Fatorando $Y(s)$:

$$Y(s).(LCs^2 + RCs + 1) = U(s)$$

Portanto a função de transferência é:

$$G(s) = Y(s)/U(s) = 1 / (LCs^2 + RCs + 1)$$

Dividindo numerador e denominador por LC pra deixar na forma canônica:

$$G(s) = (1/LC) / (s^2 + (R/L)s + 1/LC)$$

Comparando com a forma padrão $G(s) = \omega_n^2 / (s^2 + 2\zeta\omega_n s + \omega_n^2)$, obtemos:

$$\omega_n = 1/\sqrt{LC}$$

$$\zeta = (R/2).\sqrt{C/L}$$

2.4 Polos do Sistema

Os polos são as raízes do denominador:

$$s^2 + (R/L)s + 1/LC = 0$$

$$s_{1,2} = -(R/2L) \pm \sqrt{[(R/2L)^2 - 1/LC]}$$

2.5 Valores Numéricos Utilizados

Parâmetro	Valor
R	10Ω
L	0,5 H
C	0,001 F
ω_n	44,7 rad/s
ζ	0,22

3. Motor de Corrente Contínua (Motor DC)

3.1 Descrição Física

O motor DC de excitação independente é um dos atuadores mais usados em sistemas de controle. Ele converte energia elétrica em energia mecânica. O modelo considera dois subsistemas acoplados:

- Circuito elétrico da armadura: resistência R_a , indutância L_a e a força contra-eletromotriz (f_{cem}) gerada pela rotação
- Parte mecânica: momento de inércia J do rotor e coeficiente de atrito viscoso b

3.2 Esquações Diferenciais

Equação elétrica (Lei de Kirchhoff na malha da armadura):

$$v_a(t) = R_a.i_a(t) + L_a.(di_a/dt) + e_b(t)$$

Equação mecânica (Segunda Lei de Newton rotacional):

$$J.(d\omega/dt) + b.\omega(t) = T(t) = K_m.i_a(t)$$

3.3 Função Transferência

Aplicando Laplace nas duas equações:

$$V_a(s) = (R_a + L_a.s).I_a(s) + K_b.\Omega(s)$$

$$(J.s + b).\Omega(s) = K_m.I_a(s)$$

Substituindo na primeira e isolando $\Omega(s)/V_a(s)$:

$$G(s) = \Omega(s)/V_a(s) = K_m / [(L_a.s + R_a)(J.s + b) + K_m.K_b]$$

Expandindo o denominador:

$$G(s) = K_m / [L_a.J.s^2 + (R_a.J + L_a.b).s + (R_a.b + K_m.K_b)]$$

3.4 Simplificando na Prática

Na maioria dos motores DC de pequeno porte, a indutância L_a é pequena o suficiente para ser desprezada ($L_a \approx 0$). O modelo simplifica para:

$$G(s) = K_m / [R_a.J.s + (R_a.b + K_m.K_b)]$$

Dividindo por $(R_a.b + K_m.K_b)$:

$$G(s) = (K_m/R_a.b + K_m.K_b) / (\tau_m.s + 1)$$

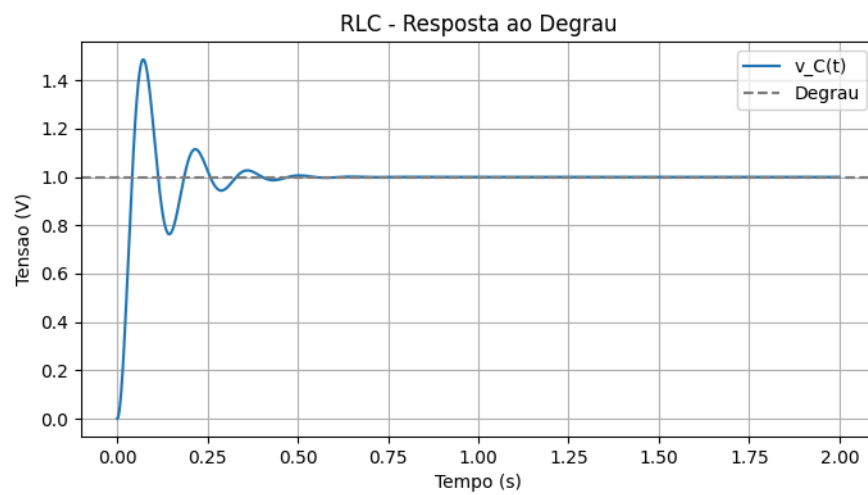
3.4 Valores Numéricos Utilizados

Parâmetro	Valor
La	1 Ω
La	0,5H
J	0,01 kg·m ²
b	0,1 N·m·s/rad
Kb	0,01 V·s/rad

4. Simulações

As simulações foram implementadas em Python com as bibliotecas NumPy, SciPy e Matplotlib. A seguir são apresentados os códigos utilizados

4.1 Circuito RLC



codigo:

```
import numpy as np

from scipy import signal

import matplotlib.pyplot as plt

# Parametros do circuito RLC

R, L, C = 10, 0.5, 0.001 # ohm, Henry, Farad

# Funcao de transferencia:  $G(s) = (1/LC) / (s^2 + (R/L)s + 1/LC)$ 

num = [1/(L*C)]

den = [1, R/L, 1/(L*C)]

sys_rlc = signal.TransferFunction(num, den)

# Frequencia natural e coeficiente de amortecimento

wn = 1 / np.sqrt(L*C)

zeta = (R/2) * np.sqrt(C/L)

print(f"wn = {wn:.2f} rad/s | zeta = {zeta:.3f}")

# Resposta ao degrau

t = np.linspace(0, 2, 2000)

t_out, y_out = signal.step(sys_rlc, T=t)

# Polos

polos = np.roots(den)

print(f"Polos: {polos}")

# Grafico

plt.figure(figsize=(8, 4))

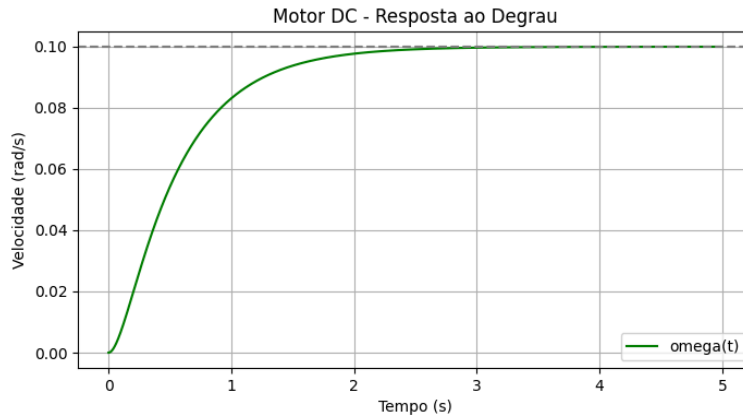
plt.plot(t_out, y_out, label='v_C(t)')

plt.axhline(1.0, ls='--', color='gray', label='Degrau')

plt.xlabel('Tempo (s)'); plt.ylabel('Tensao (V)')

plt.title('RLC - Resposta ao Degrau'); plt.legend(); plt.grid(); plt.show()
```

4.2 Motor DC



código:

```
import numpy as np

from scipy import signal

import matplotlib.pyplot as plt

# Parametros do Motor DC

Ra=1; La=0.5; J=0.01; b=0.1; Km=0.01; Kb=0.01

num_dc = [Km]

den_dc = [La*J, Ra*J + La*b, Ra*b + Km*Kb]

sys_dc = signal.TransferFunction(num_dc, den_dc)

# Polos

polos = np.roots(den_dc)

print(f'Polos Motor DC: {polos}')

# Resposta ao degrau

t = np.linspace(0, 5, 3000)

_, y_dc = signal.step(sys_dc, T=t)

tau_m = Ra*J / (Ra*b + Km*Kb)

print(f'Constante de tempo mecanica: tau_m = {tau_m:.4f} s')

plt.figure(figsize=(8, 4))
```

```
plt.plot(t, y_dc, color='green', label='omega(t)')

plt.axhline(y_dc[-1], ls='--', color='gray')

plt.xlabel('Tempo (s)'); plt.ylabel('Velocidade (rad/s)')

plt.title('Motor DC - Resposta ao Degrau'); plt.legend(); plt.grid(); plt.show()
```

5. Análise dos Resultados

Circuito RLC: com $\zeta = 0,22$ o sistema oscila bastante antes de estabilizar. Aumentando R aumenta o amortecimento e reduz as oscilações. Isso faz sentido fisicamente: o resistor dissipa energia, enquanto o indutor e o capacitor ficam trocando energia entre si causando a oscilação.

Quando se varia C, ω_n muda mas ζ também é afetado, os dois parâmetros não são independentes. Isso é uma limitação do circuito RLC: não dá pra ajustar velocidade e amortecimento separadamente sem mexer nos dois componentes ao mesmo tempo.

Motor DC: os dois polos reais negativos confirmam o comportamento superamortecido, a velocidade sobe sem oscilar, o que é desejável na maioria das aplicações práticas. O polo dominante é o $s_1 \approx -2$, que é o mais lento e define o tempo de resposta do motor.

Ao aumentar a inércia J os polos se aproximam da origem (resposta mais lenta). Ao aumentar o atrito B o polo mais lento se afasta da origem (resposta mais rápida, mas com perda de eficiência). Isso mostra que no motor DC dá pra ter uma intuição física boa sobre o efeito de cada parâmetro.

Comparando os dois sistemas: o RLC com baixo amortecimento tem resposta mais rápida mas com sobressinal. O motor DC superamortecido é mais lento mas sem oscilação. Em controle, a escolha depende da aplicação, sistemas de posicionamento de precisão preferem menos oscilação, enquanto sistemas que precisam de resposta rápida podem tolerar algum sobressinal.

6. Aplicações Práticas

O modelo do circuito RLC aparece diretamente em filtros eletrônicos de segunda ordem, como os filtros passa-baixa usados em fontes de alimentação e em sistemas de áudio. Também é a base para modelar circuitos de compensação em malhas de controle.

O modelo do motor DC é amplamente usado em robótica industrial, acionamentos de CNCs, impressoras 3D e qualquer sistema que precise controlar velocidade ou posição de um eixo. No controle de posição, um integrador é adicionado ao modelo do motor